

We use some essential cookies to make our website work.

We use optional cookies, as detailed in our **cookie policy**, to remember your settings and understand how you use our website.

Accept optional cookies

Reject optional cookies

Documentation



Computers

Accessories

Microcontrollers

Silicon

Pico-series

Microcontrollers

Raspberry Pi
Debug Probe

MicroPython

**The C/C++
SDK**

**SDK
Setup**

Raspberry
Pi Pico C/
C++ SDK

Your
First
Binaries

Quick-
start
your own
project

Services

Pico C SDK

Datasheets

Product
Information
Portal

Tutorials

–

The C/C++ SDK

SDK Setup

[Edit this on GitHub](#)

For a full walk-through of how to get going with the C/C++ SDK, you should read our 'getting started' documentation. However, if you are intending to develop for Pico on a [Raspberry Pi](#), then you can set up the C/C++ toolchain quickly by running our [setup script](#) from the command line.

NOTE

You should make sure the OS on your Raspberry Pi is [up to date](#) before running the setup script.

Raspberry Pi Pico C/C++ SDK

[Edit this on GitHub](#)

Our official C SDK can be used from the command line, or from popular integrated development environments like Visual Studio Code, Eclipse, and CLion. To get started, download our C/C++ SDK and Examples, and take a look at our 'getting started' documentation. Or for a quick setup see the next section.

You can find documentation around the C/C++ SDK at:

Getting started with Raspberry Pi Pico-series

C/C++ development with Raspberry Pi Pico, Pico 2, and other Raspberry Pi microcontroller-based boards

Connecting to the Internet with Raspberry Pi Pico W

Getting Raspberry Pi Pico W online with C/C++ or MicroPython

Raspberry Pi Pico-series C/C++ SDK

Libraries and tools for C/C++ development on Raspberry Pi microcontrollers

API level documentation

Documentation for the Raspberry Pi Pico C/C++ SDK

The pico-examples repository

Example projects

SDK source code is open source, available via the [pico-sdk Github repository](#).

NOTE

To build applications with the C/C++ SDK for a board *other than the Raspberry Pi Pico*, pass `-DPICO_BOARD=boardname` to CMake, where `boardname` is the name of your board. For example:

- to build an application for the Adafruit Feather RP2040, pass –
`DPICO_BOARD=adafruit_feather_rp2040`
- to build an application for Pico W, pass `-DPICO_BOARD=pico_w` (in addition to `-DWIFI_SSID="Your Network" -DWIFI_PASSWORD="Your Password"`, should you wish to connect to a wireless network)

For more information, see the [boards/ directory](#) in the Raspberry Pi Pico SDK and the [forums](#).

On this page

[SDK Setup](#)

[Raspberry Pi Pico C/C++](#)

[SDK](#)

[Your First Binaries](#)

[Quick-start your own project](#)

Your First Binaries

[Edit this on GitHub](#)

Blink an LED

The first program anyone writes when using a new microcontroller is to blink an LED on and off. The Raspberry Pi Pico comes with a single LED on-board. The LED is connected to **GP25** on the board's Raspberry Pi RP2040 for Pico, and **WL_GPI00** on the Infineon 43439 wireless chip for Pico W.



0:00 / 0:25

You can blink this on and off by,

1. Download the Blink UF2 [for Raspberry Pi Pico](#), or [for Pico W](#).
2. Push and hold the BOOTSEL button and plug your Pico into the USB port of your Raspberry Pi or other computer.
3. It will mount as a Mass Storage Device called RPI-RP2.
4. Drag and drop the Blink UF2 binary onto the RPI-RP2 volume. Pico will reboot.

You should see the on-board LED blinking.

You can see the code on Github for the [Raspberry Pi Pico](#) and [Pico W](#) versions.

Say "Hello World"

The next program anyone writes is to say 'Hello World' over a USB serial connection.



0:00 / 0:26

1. Download the 'Hello World' UF2.
2. Push and hold the BOOTSEL button and plug your Pico into the USB port of your Raspberry Pi or other computer.
3. It will mount as a Mass Storage Device called RPI-RP2.
4. Drag and drop the 'Hello World' UF2 binary onto the RPI-RP2 volume. Pico will reboot.
5. Open a Terminal window and type:

```
$ sudo apt install minicom
$ minicom -b 115200 -o -D /dev/ttyACM0
```

You should see 'Hello, world!' printed to the Terminal.

You can see the code [on Github](#)

Quick-start your own project

Edit this [on GitHub](#)

NOTE

The following instructions are terse, and Linux-based only. For detailed steps, instructions for other platforms, and just in general, we recommend you see the [Getting started with Raspberry Pi Pico](#) and [Raspberry Pi Pico C/C++ SDK](#) books.

Install CMake (at least version 3.13), and GCC cross compiler

```
$ sudo apt install cmake gcc-arm-none-eabi libnewlib-arm-none-eabi libstdc++-arm-none-eabi-newlib
```

Set up your project to point to use the Raspberry Pi Pico SDK by cloning the SDK locally:

```
$ git clone https://github.com/raspberrypi/pico-sdk.git
```

Copy `external/pico_sdk_import.cmake` from the SDK into your project directory

Set `PICO_SDK_PATH` to the SDK location in your environment, or pass it (`-DPICO_SDK_PATH=`) to `cmake` later.

Setup a `CMakeLists.txt` like:

```
cmake_minimum_required(VERSION 3.13)

# initialize the SDK based on PICO_SDK_PATH
# note: this must happen before project()
include(pico_sdk_import.cmake)

project(my_project)

# initialize the Raspberry Pi Pico SDK
pico_sdk_init()

# rest of your project
```

Go ahead and write your code, see [pico-examples](#) or the [Raspberry Pi Pico C/C++ SDK](#) book for more information on how to go about that.

About the simplest you can do is a single source file (e.g. `hello_world.c`)

```
#include <stdio.h>
#include "pico/stdlib.h"

int main() {
    setup_default_uart();
    printf("Hello, world!\n");
    return 0;
}
```

and add the following to your `CMakeLists.txt`:

```
add_executable(hello_world
    hello_world.c
)

# Add pico_stdlib library which aggregates commonly used features
target_link_libraries(hello_world pico_stdlib)

# create map/bin/hex/uf2 file in addition to ELF.
pico_add_extra_outputs(hello_world)
```

NOTE

This example uses the default UART for stdout; if you want to use the default USB see the [hello-usb](#) example.

Setup a CMake build directory. For example, if not using an IDE:

```
$ mkdir build
$ cd build
$ cmake ..
```

When building for a board other than the Raspberry Pi Pico, you should pass `-DPICO_BOARD=board_name` to the cmake command above, e.g. `cmake -DPICO_BOARD=pico_w ..` to configure the SDK and build options accordingly for that particular board.

Doing so sets up various compiler defines (e.g. default pin numbers for UART and other hardware) and in certain cases also enables the use of additional libraries (e.g. wireless support when building for `PICO_BOARD=pico_w`) which cannot be built without a board which provides the requisite functionality.

For a list of boards defined in the SDK itself, look in [this directory](#) which has a header for each named board.

Make your target from the build directory you created.

```
$ make hello_world
```

You now have `hello_world.elf` to load via a debugger, or `hello_world.uf2` that can be installed and run on your Raspberry Pi Pico via drag and drop.

You can view and edit the Raspberry Pi documentation source [on Github](#). Please read our [usage and contributions policy](#) before you make a Pull Request.

Raspberry Pi documentation is copyright © 2012-2025 Raspberry Pi Ltd and is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International](#) (CC BY-SA) licence.

Some content originates from the [eLinux wiki](#), and is licensed under a [Creative Commons Attribution-ShareAlike 3.0 Unported](#) licence.

The terms HDMI, HDMI High-Definition Multimedia Interface, HDMI trade dress and the HDMI Logos are trademarks or registered trademarks of HDMI Licensing Administrator, Inc